

University at Buffalo
Department of Computer Science and Engineering
CSE 473/573 - Computer Vision and Image Processing

Project #3
Due Date: 12/04/25, 11:59PM

1 Task 1: Face detection (3 pts)

Given a face detection dataset composed of hundreds of images, the goal is to detect faces contained in the images. Each detector should be able to locate the faces in any testing image. Figure 1 shows an example of performing face detection. We will use a subset (will be provided) of FDDB [1] as the dataset for this project. You can use any face detection modules available in face recognition.



Figure 1: An example of performing face detection. The detected faces are annotated using gray bounding boxes.

1.1 Additional information

- You will be given a folder contains 100 images along with ground-truth annotations (validation folder).
- You can evaluate each of your model's performances on this validation set or use it to further improve your detection. During testing, you need to report results on another 100 images (test folder) without ground truth annotations.

- Please read the README.md files provided in the project repo and refer to the script in it for running and validating your code.
- Your implementation should be in the function **detect_faces()**, in the file face.py.
- The function should detect faces in the given image and return the bounding boxes of the detected faces in a list (maybe more than 1 face in an image). The bounding box should be in the format of [x, y, width, height], x and y are the top-left corners of the bounding box; width and height are the width and height of the bounding box, respectively. x, y, width, and height should be float numbers. Consider origin (0,0) to be the top-left corner of the image and x increases to the right and y increases to the bottom. See the code for the rest of the input and output formats.

2 Task 2: Face clustering (7 pts)

You will be building on top of the above face detection code to do face clustering. You will be using images present in the faceCluster K folder for part B (i.e., face clustering). Each image will only contain one face in it. K in faceCluster_K folder provides you with the unique number of face clusters present.

2.1 Steps Involved.

- Use the function `face_recognition.face_encodings(img, boxes)` to get a 128-dimensional vector for each cropped face.
- **img** is the image.
- **boxes** is a list of found face locations. Each face location is a tuple (top, right, bottom, left). `top = y`, `left = x`, `bottom = y + height`, `right = x + width`. So, boxes would be something like [(top, right, bottom, left)]. **face_recognition.face_encodings(img, boxes)** would return a list of 128 dimensions numpy.ndarray or 2D List or torch.Tensor for each face present in the image 'img'.
- Using these computed face vectors, you need to code a k-means or other relevant clustering algorithm. If you use K-Means, or another algorithm requiring a pre-defined number of clusters, that number will be K from faceCluster_K. **You have to implement the clustering algorithm by yourself.** You may not use OpenCV or any library APIs that have 'face', 'kmeans', 'knn', or 'cluster' in their name or any other libraries which have to implement the clustering algorithm for you.
- Your implementation should be in the function **cluster_faces()**, in the file face.py.

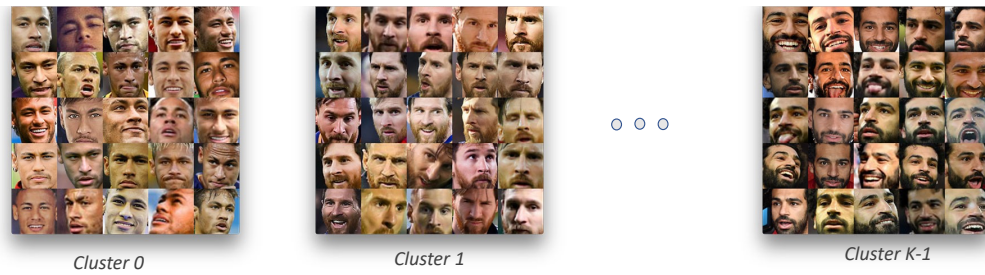


Figure 2: Face clusters.

3 Evaluation Rubric

- Task 1: We will provide ComputeFBeta.py, a python code that computes f-beta using detection results and ground truth. For F1 score computed using ComputeFBeta.py.
YourScore = $\min(3, 3.75 * F1)$
- Task 2: Accuracy will be based on the number of faces with the same identity present in a cluster.
YourScore = $\min(7, 8.75 * F1)$

4 Instructions and Submission Guidelines

- For the main task: Please implement all your code in the file "**face.py**".
- Please do NOT make any changes to any file except "**face.py**".
- Please do NOT read/write any files in your code. The file reading and output part are already given in "**task1.py**" and "**task2.py**". In your implementation, you should ONLY use the input parameters of the function and give the output in a required data structure. The data structure is given in the code.
- Please do not use cv2, numpy, PIL, imageio for any image-related operations. We have provided the required helper functions. If you see any custom methods being useful for your implementation, then please feel free to implement them with PyTorch.
- You can only use the given libraries provided in the code. You can not make any new imports.
- To submit your code and result, Please run "**pack_submission.sh**" to pack your code and result into a zip file. You can find the command line in "**README.md**". Note that when packing your submission, the script would run your code before packing. The resulting zip file is the only file you need to submit.
- The packed submission file should be named "**submission_YourUBITName.zip**", and it should contain 7 files, named "**result_task1.json**", "**result_task1_val.json**", "**result_task2.json**", "**face.py**". If not, there is something wrong with your code/filename, please go back and check.

- For code raising “RuntimeError”, the grade will be ZERO for the project if it can not be corrected.
- Identical code will be treated as plagiarism. Please work it out independently. Checks will be done for AI violations. If your submission is evaluated as plagiarism, your grade will be 0 for whole project.
- Unlimited number of submissions is allowed and only the latest submission will be used for grading.

5 Preparing for the environment

- **Please note:** In this project, it is highly recommend to use anaconda to manage your environment. Here is a link about miniconda usage: "<https://docs.anaconda.com/free/miniconda/index.html>"
- If you are using windows, We also **strongly recommend** to use **Windows Subsystem for Linux** ("<https://learn.microsoft.com/en-us/windows/wsl/install>") instead of install directly. If you meet problem in installing the face_recognition, please try it first.

5.1 Install face_recognition

- Please make sure you have cmake and build-essential installed. If not please try following command:

```
$ sudo apt-get update
$ sudo apt-get install cmake
$ sudo apt-get install build-essential
```

- You need to install dlib before you install face_recognition

```
$ git clone https://github.com/davisking/dlib.git
$ cd dlib
$ mkdir build; cd build; cmake ..; cmake --build .
$ cd ..
$ python3 setup.py install
```

- Then you can install the face_recognition

```
$ pip3 install face_recognition
```

- For more details it can be found from following url:
- https://github.com/ageitgey/face_recognition

6 References

[1] V. Jain and E. L. Miller, “Fddb: a benchmark for face detection in unconstrained settings,” 2010.